Appendix
ONNX and TOSA based Portability and Rapidus / Tenstorrent Inference Architecture

Prepared by Akira on Dec. 04, 2025

The proposed architecture separates model provenance from the inference hardware ecosystem. Models may originate from NVIDIA based training, AWS Trainium, or other training environments. Regardless of their training origin, a model can enter the proposed inference pipeline when its computational graph is representable within a defined standard ONNX operator subset.

The model is first represented using the standard ONNX operator subset and then lowered to TOSA. In this architecture, ONNX serves as the model level interchange representation, while TOSA serves as the compiler level intermediate representation. TOSA does not eliminate the need for hardware specific optimization; rather, it provides a common computational representation from which target specific optimization and lowering can be performed.

The resulting pipeline is:
NVIDIA / Trainium: AWS / other training platform
→ standard ONNX operator subset
→ TOSA
→ hardware specific optimization and lowering
→ Rapidus/Tenstorrent inference architecture

This architecture therefore does not require an NVIDIA trained model to remain dependent on NVIDIA hardware or software during inference. If the model can be expressed within the supported ONNX operator subset, it can be lowered through TOSA and subsequently optimized for the Rapidus/Tenstorrent inference architecture.

The same principle applies to models trained on AWS Trainium. Trainium can serve as the designated training platform without making Trainium the mandatory inference platform. The training environment and the inference environment are consequently treated as separate architectural decisions.

TOSA provides an additional layer of abstraction between the standardized model representation and the physical inference hardware. It does not, however, eliminate hardware specific compilation or optimization. After conversion to TOSA, the computational graph still has to be optimized and lowered according to the

characteristics of the target hardware and its execution environment. In the proposed architecture, this final stage is directed toward the Rapidus/Tenstorrent inference architecture.

The NVIDIA trained model case is particularly important as a practical test of the proposed separation between model provenance and inference hardware. If a model originally trained on NVIDIA hardware can be represented within the supported ONNX operator subset, lowered to TOSA, and subsequently optimized for Rapidus/Tenstorrent inference, then the inference deployment is no longer intrinsically determined by the hardware ecosystem in which the model was trained.

The objective is not unrestricted hardware portability. Instead, the architecture provides controlled portability at the model and compiler representation layers while retaining hardware specific optimization at the final stage. ONNX provides the model level interchange boundary, TOSA provides the compiler level abstraction, and the hardware specific backend layer performs the optimization and lowering required for efficient edge inference.

This layered architecture allows models originating from different training ecosystems, including NVIDIA, to be redirected toward the Rapidus/Tenstorrent inference architecture without requiring the inference environment to reproduce the original training hardware and software stack.

＊＊＊＊＊＊＊

Note-1: Models containing unsupported operators would require additional conversion or adaptation before entering the proposed pipeline.

＊＊＊＊＊＊＊

Note-2: Portability of the computational representation does not by itself guarantee equivalent performance across different hardware architectures. Hardware specific optimization remains necessary to achieve efficient inference on the target architecture.

＊＊＊＊＊＊＊ A Detailed Example of Hardware Specific Optimization ＊＊＊＊＊＊＊

The trained model is passed as ONNX, and that ONNX representation is lowered to TOSA. Here, ONNX functions as the boundary for model exchange and representation, while TOSA functions as a common tensor-level representation handled on the compiler side. Therefore, at the TOSA stage, the computations performed by the model are represented, but how those computations are to be executed concretely on a specific accelerator has not yet been determined. Actual performance is determined by what comes after TOSA.

After TOSA, Tenstorrent's Compiler, Backend, and Runtime control and optimize how the TOSA representation is executed on the target NN accelerator, while the Rapidus side is responsible for the design and implementation of the NN accelerator hardware. These are linked through HW/SW co-design to transform the computations represented in TOSA into a form that can be executed efficiently on the actual NN accelerator.

What is important here is that this is not simply a matter of converting TOSA into hardware-specific instructions. The computational graph of the model itself is analyzed, and the execution method suited to the structure of the target hardware is determined while mutually adjusting operator fusion, tensor partitioning, memory placement, data layout, numerical precision, execution order, kernel generation, and so on.

Specifically, it is determined which PE performs the computation, into which tiles the computation is divided, and where those tiles are placed in SRAM. Furthermore, it is determined when the necessary data is Loaded, when it is passed to Compute, and when the computation results are Stored, while also designing which data is reused within SRAM. In addition, the precision at which each operation is executed is selected, and among consecutive operators, it is determined which operations are fused and executed as a single process. In other words, rather than executing as-is the abstract computational content of <what to compute> represented in TOSA, it is "lowered into" a concrete execution method according to the PE, tiles, SRAM, data movement paths, computational precision, and operator configuration of the target accelerator: where, in what units, where to place the data, in what order, and at what precision to perform the computation.

In Graph optimization, the entire computational graph of the model is analyzed and the graph is transformed so that unnecessary intermediate processing and data movement can be reduced. Here, individual operators are not considered in isolation, rather, the relationships among preceding and subsequent operations and the dependencies among tensors are also taken into account to determine what execution form is efficient overall.

In Operator fusion, multiple consecutive operations are fused into a single execution unit. For example, when operations such as MatMul, Bias, and Activation are executed separately, the intermediate tensor generated between them may need to be written to memory and then read again. If these operations can be fused, memory access for intermediate data and the overhead of kernel launches can be reduced. Therefore, in an NN accelerator, not only the computations themselves but also how much data movement between operations can be reduced is important.

In Tiling, large tensors or matrices are divided into tiles of a size that the accelerator

can process efficiently. The size of the division depends on the accelerator's PE configuration, the data width of the arithmetic units, the capacity of local SRAM, the data transfer mechanism, and so on. Thus, Tiling is the first important stage for adapting the abstract TOSA representation to a concrete hardware structure.

In Memory planning, it is determined in which memory regions each tensor or tile is placed, when it is loaded, how long it is retained, and when it is released. In particular, in an NN accelerator with local SRAM, keeping the data required for computation in nearby memory and reusing it can reduce data transfers with external memory. Therefore, the decision of <which data to reuse> is not merely a matter of memory capacity, but an important optimization that determines the effective performance of the accelerator.

In Layout transformation, the form in which tensors are placed in memory is transformed. Even when the tensor is mathematically the same, memory access efficiency and the efficiency of supplying data to the PEs change depending on how the data is arranged. If the target accelerator assumes tile-based processing or a specific data arrangement, it is necessary to transform the data into a layout that allows the hardware to acquire and process the data as efficiently as possible.

In Precision selection, the numerical precision to be used for each operation is determined. The selection of FP32, BF16, FP16, FP8, INT8, and so on changes computational performance, memory capacity, memory bandwidth, power consumption, and model accuracy. Accordingly, rather than simply lowering precision, it is necessary to determine it by considering the balance between the computational capabilities provided by the target hardware and the precision required by the model.

In Kernel/code generation, the graph structure, operator fusion, tile partitioning, memory placement, layout, precision, and so on that have been determined up to that point are lowered into kernels or code that can actually be executed by the Tenstorrent accelerator. Here, operations such as MatMul and Conv represented abstractly in TOSA are transformed into forms that execute using concrete hardware resources.
The decision of which computations are assigned to which PEs or computational resources is also closely connected with the various optimizations up to this stage.

In Scheduling, it is determined when the generated processes are executed. Rather than simply performing Load, Compute, and Store sequentially, data transfers and computations are overlapped as much as possible, and the execution order is constructed so that resources such as PEs, memory, and the network do not have idle waiting time. Decisions about when to operate each PE, when to supply each tile, and

when to pass computation results to the next process directly affect the utilization of the accelerator as a whole.

Runtime is the layer that manages and executes the execution plan created by the compiler on the actual device. It handles kernel dispatch, buffer and tensor management, data transfer, synchronization, queue management, and so on, and makes the execution method determined at compile time actually work on the accelerator.

Viewed in this way, the optimization processes below TOSA are not merely <conversion processes>. Each stage from Graph optimization through Runtime is mutually related to the PE configuration, tile size, SRAM capacity and placement, data movement paths, computational precision, kernels, and execution order. As a result, even if TOSA is adopted as a common tensor-level representation, the optimization after that point strongly depends on the structure of the target accelerator.

For this reason, it is meaningful to connect Tenstorrent's Compiler, Backend, and Runtime with the NN accelerator hardware design on the Rapidus side through HW/SW co-design.

The compiler side determines the dataflow, tile structure, memory placement, and execution schedule needed for efficient NN processing, and these are reflected in the design of the PEs, SRAM, NoC, memory hierarchy, and arithmetic circuits on the hardware side. Conversely, the structure and constraints on the hardware side are fed back to the Compiler side and reflected in graph optimization, tiling, memory planning, kernel generation, scheduling, and so on that make maximum use of that structure.

Taken together, the technical core of this concept is not simply that ONNX can be converted to TOSA. Rather, it lies in establishing TOSA as a common computational representation, then integrating the Compiler, Backend, and Runtime beneath it with the design of the NN accelerator so that the computations represented in TOSA can be executed as efficiently as possible on specific hardware.

In other words, by standardizing model representation through ONNX and TOSA, and conversely strengthening the coupling with hardware below TOSA, it becomes possible to accept trained models as a common representation while, for inference execution, thoroughly optimizing them for a specific NN accelerator by combining Tenstorrent's Compiler/Backend/Runtime with Rapidus's chip design.

＊＊＊＊＊＊＊

Note-3: SYCL (see Section No.1 in the main paper) centered open source compiler ecosystem, combined with TOSA based portable computational representation, can

support vendor independent portability while allowing hardware specific optimization for the Rapidus GAA architecture.

Note-4: ONNX ; Open Neural Network Exchange,
TOSA ; Tensor Operator Set Architecture
OpenCL: Low-level parallel programming centered on the C language
SYCL: A single source C++ programming model built on OpenCL

****

With best wishes for your continued research and development.

Akira Ver.1.4-3 Revised July 30, 2026